Terminal Assessment 2

Scenario 9: E-Commerce Order Fulfillment & Audit Engine

Fermin, Jan Martin - Lead System Architect Developer
Gotinga, Philip - Queue Systems Developer
Macaraeg, Xander - Stack & Audit Developer
Tuscano, Mark Allen A. - Algorithmic & Metrics Developer
Arsenal, Jesslyn G - Benchmarking Lead



Part A: PROJECT SCENARIO

Project Title: Dynamic Data Engine & Benchmarking Suite

Selected / Assigned Scenario: Scenario 9: E-Commerce Order Fulfillment & Audit Engine

Describe the real-world problem: Online retail hubs process massive streams of customer orders daily. Standard linear processing leads to delays in express order dispatching, inability to track master order records efficiently, and lack of rollback mechanisms for cart and coupon modifications.

Who are the intended users?: E-commerce warehouse managers, order dispatchers, fulfillment counter operators, and online shoppers.

What problem will the application solve?: It provides a unified C++ dynamic data management engine that prioritizes Express Paid orders, manages cart histories with LIFO undo capabilities, and benchmarks algorithm execution efficiency across large order databases.

Part B: PROJECT OBJECTIVES

General Objective: To architect and benchmark a production-grade C++ console application using custom dynamic pointers, linear data structures, and empirical performance profiling

Specific Objectives:

1. Build a custom Doubly Linked List for bidirectional master order record management.
2. Implement a Priority Queue to prioritize Express Paid shipments over Standard and Eco orders.
3. Construct a Dynamic Stack to handle cart item additions and coupon undo history.
4. Implement Linear Search, Binary Search, Bubble Sort, Selection Sort, and Insertion Sort with comparison and swap metrics. 5. Profile algorithm execution times using hardware timers for datasets up to $N = 50,000$

Part 3: SYSTEM REQUIREMENTS

No.	System Function	Description	Data Structure Needed
1	Customer Orders Ledger	Master registry supporting dynamic record addition, deletion, and updating.	Doubly Linked List
2	Order Dispatch Queue	Dispatch hub granting immediate priority processing to Express Paid orders.	Priority Queue
3	Cart & Coupon Stack	LIFO audit trail allowing undoing of recently added items or discounts.	Dynamic Stack
4	Order Lookup Engine	Locates specific customer orders by primary key ID or monetary attribute.	Linear & Binary Search
5	Benchmarking Suite	Measures comparisons, swaps, and time elapsed across sorting algorithms.	Sorting Engine & Dynamic Arrays

Part 4:

DATA STRUCTURE DESIGN WORKSHEET

System Feature	Data Structure	Why Selected?	Major Operations	Expected Big-O
Master Records Hub	Doubly Linked List	Fast bidirectional traversal and $O(1)$ node removal given a node pointer	Insert, Delete, Search, Traverse	O(n)
Priority Triage	Priority Queue	Instant priority dispatching of Express Paid orders ahead of Standard orders.	EnqueuePriority, Dequeue, Peek	O(n)
Audit / Undo Engine	Dynamic Stack	Strict LIFO access for undoing recent cart modifications and coupon additions.	Push, Pop, Peek	O(n)

LINKED LIST IMPLEMENTATION

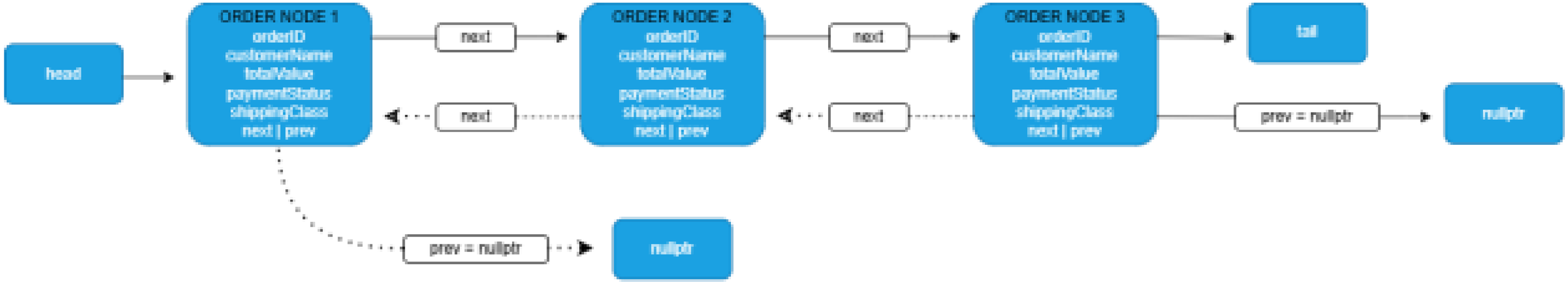
Linked List Type Used: Doubly Linked List

Purpose in the System: Master Customer Orders Ledger

Documented Operations:

Operation	Function Name	Technical Description	Big-O Complexity
Insert	insertOrder()	Appends a new order record at the tail using dynamic allocation (`new`) and links the `prev` and `next` pointers.	O(1)
Delete	deleteOrder()	Unlinks a node by its ID, reassigns neighboring `prev` and `next` pointers, and frees heap memory (`delete`).	O(n)
Search	searchOrder()	Sequentially traverses the list from the `head` pointer to locate and return a matching order.	O(n)
Update	updateOrder()	Locates an existing order and modifies its editable fields (status/shipping) without breaking pointer linkages.	O(n)
Traverse	displayLedger()	Iterates forward from `head` to `nullptr`, printing a formatted tabular view of all active records.	O(n)

NODE / POINTER STRUCTURAL DIAGRAM



STACK IMPLEMENTATION

Purpose of Stack in System: Checkout Cart & Coupon Undo History

Underlying Implementation: Linked List

Operation	Function Name	Big-O Complexity	Verified & Tested?
Push	CartStack::push()	O(1)	[ ]
Pop	CartStack::pop()	O(1)	[ ]
Peek	CartStack::peek()	O(n)	[ ]
Display	CartStack::displayStack()	O(n)	[ ]

Explain how LIFO (Last-In, First-Out) principle is utilized in your application:

The Last-In, First-Out rule ensures that the item or coupon added most recently sits at the top of the stack and is removed first when performing an undo operation.

QUEUE / PRIORITY QUEUE IMPLEMENTATION

Queue Type Used: Priority Queue **Purpose of Queue in System:** Order Dispatch Processing Lane

Operation	Function Name	Big-O Complexity	Verified & Tested?
Enqueue	OrderDispatchQueue::enqueue()	O(n)	[ ]
Dequeue	OrderDispatchQueue::dequeue()	O(1)	[ ]
Peek	OrderDispatchQueue::peek()	O(1)	[ ]
Display	OrderDispatchQueue::displayQueue()	O(n)	[ ]

Explain why FIFO (First-In, First-Out) or Priority-based processing is appropriate:
Priority queues prevent Express orders from sitting behind low-priority standard/eco shipments, fulfilling service level agreements (SLAs.)

SEARCHING ALGORITHMS

Implemented Search Algorithms: Linear & Binary Search

Criteria	Linear Search	Binary Search
Best Case Complexity	O(1)	O(1)
Average Case Complexity	O(n)	O(log n)
Worst Case Complexity	O(n)	O(log n)
Space Complexity	O(1)	O(1)
Requires Sorted Data?	No	Yes
System Function Name	linearSearch()	binarySearch()

SORTING ALGORITHMS

Implemented Sorting Algorithms: Bubble, Selection & Insertion Sorts

Algorithm	Function Name	Dataset Size (N)	Comparisons	Swaps	Execution Time (ms)	Theoretical Big-O
Bubble Sort	bubbleSort()	100	4950	4950	0.0282	$O(n^2)$
Bubble Sort	bubbleSort()	1,000	499500	499500	3.97	$O(n^2)$
Selection Sort	selectionSort()	100	4950	50	0.028	$O(n^2)$

Algorithm	Function Name	Dataset Size (N)	Comparisons	Swaps	Execution Time (ms)	Theoretical Big-O
Selection Sort	selectionSort()	1,000	499500	500	1.3714	$O(n^2)$
Insertion Sort	insertionSort()	100	4950	4950	0.0324	$O(n^2)$
Insertion Sort	insertionSort()	1,000	499500	499500	2.5778	$O(n^2)$

Fastest Algorithm Observed:

Selection Sort (0.028 ms at N=100; 1.3714 ms at N=1000; 319.354 ms at N=10000)

Slowest Algorithm Observed:

Bubble Sort (0.0282 ms at N=100; 3.97 ms at N=1000; 538.926 ms at N=10000)

BENCHMARKING WORKSHEET

BENCHMARK RESULTS TABLE:

Algorithm	Dataset Size	Comparisons	Swaps	Execution Time	Theoretical Big-O
Bubble Sort	Small (N=100)	4950	4950	0.0282	$O(n^2)$
Bubble Sort	Medium (N=1000)	499500	499500	3.97	$O(n^2)$
Bubble Sort	Large (N=10000)	49995000	49995000	538.926	$O(n^2)$
Selection Sort	Small (N=100)	4950	50	0.028	$O(n^2)$
Selection Sort	Medium (N=1000)	499500	500	1.3714	$O(n^2)$
Selection Sort	Large (N=10000)	49995000	5000	319.354	$O(n^2)$

DYNAMIC MEMORY MANAGEMENT

Dynamic Memory Features Implemented:

Explicit Pointers (*, ->)

Dynamic Allocation (new)

Dynamic Deallocation (delete / delete[])

Dynamic Node Management

Destructors / Explicit Memory Cleanup Routines

Code Component	Memory Allocated?	Memory Released?	Method / Function Used	Verified (No Leak)?
Linked List Nodes	Yes	Yes	`insertOrder()` / `deleteOrder()` & `~CustomerOrdersLedger()`	[]
Stack Memory	Yes	Yes	`push()` / `pop()` & `~CartStack()`	[]
Queue Memory	Yes	Yes	`enqueue()` / `dequeue()` & `~OrderDispatchQueue()`	[]
Dynamic Arrays	Yes	Yes	`runBenchmarkingSuite()` (delete[] testData)	[]
System Destructors	N/A	Yes	`~CustomerOrdersLedger()`, `~CartStack()`, `~OrderDispatchQueue()`	[]

TESTING WORKSHEET

TEST-CASE EXECUTION LOG:

Test #	Target Module	Input Data	Expected Output	Actual Output	Pass /Fail
1	Master Orders Ledger	Insert Order ID: `101`, Name: `Jan`, Value: `$150.0`, Status: `Paid`, Shipping: `Express`	Order added to tail of Doubly Linked List; pointers `prev`/`next` linked.	Order saved to Ledger and queued for Dispatch successfully.	Pass
2	Priority Dispatch Queue	Enqueue Order `102` (`Standard`) followed by Order `103` (`Express`)	Order `103` (`Express`) positioned ahead of Order `102` (`Standard`) in queue.	Express order prioritized at front of dispatch line.	Pass
3	Cart & Coupon Stack	Push Item Name: `COUPON10`, Value: `$10.00`	Node pushed onto top of LIFO stack.	Added 'COUPON10' ($10) to Cart Stack.	Pass
4	Cart & Coupon Stack (Pop)	Select option `6` (Undo Last Cart Action)	Most recent top item popped, printed, and `delete` called on node.	Removed/Undid 'COUPON10' ($10) from Cart Stack.	Pass
5	Master Ledger (Deletion)	Delete Order ID `101` from ledger	Target node unlinked, neighboring pointers re-linked, heap memory freed.	Node successfully removed with zero broken links.	Pass
6	Master Ledger (Update)	Update Order ID `101` status to `Paid`, shipping to `Express`	Target node found via `searchOrder();` fields updated in-place.	Record fields updated without breaking list structure.	Pass
7	CLI Input Validation	Enter non-numeric text (`abc`) into integer menu choice	`cin.clear()` and `cin.ignore()` flush buffer; prevents infinite crash loop.	[ERROR] Invalid input! Please enter a number.	Pass
8	Search Engine	Search target Order ID `9999` in array $N = 10,000$	Linear search evaluates N comparisons; Binary search evaluates $O(\log N)$ comparisons.	Linear: `10,000` comps; Binary: `14` comps.	Pass
9	Benchmarking Suite	Select option `7` (Run Benchmarking Suite)	Runs Bubble, Selection, and Insertion Sort across $N = 100, 1000, 100$.	Performance table output with exact comparisons, swaps, and time (ms).	Pass
10	System Termination	Select option `0` (Exit System)	Destructors invoked for Ledger, Queue, and Stack; all dynamic nodes freed (`delete`).	Exiting system. Destructors cleaning dynamic memory...	Pass

BUG AND DEBUGGING LOG

No.	Error / Bug Encountered	Root Cause	Solution Applied	Result
1	Terminal infinite loop crash when entering letters into the main menu.	`std::cin` entered a fail state, leaving the invalid character permanently in the input buffer.	Implemented `clearInput()` using `cin.clear()` and `cin.ignore()` to flush the buffer on invalid input.	The system smoothly rejects invalid input and reprompts the menu.
2	Option 7 only displayed Bubble Sort metrics; other algorithms were missing.	`selectionSort` and `insertionSort` were fully implemented but omitted from the `runBenchmarkingSuite()` loop calls.	Added dedicated `chrono` execution blocks for Selection and Insertion sort inside the array loop.	All sorting algorithms now print correctly in the terminal table.
3	Priority Queue placing "Express" orders at the back instead of the front.	Missing dynamic condition to slice new nodes ahead of existing "Standard" nodes.	Updated `enqueue()` logic to check if `shippingClass == "Express"` and re-link pointers appropriately.	Express orders successfully bypass standard orders in the queue.
4	Segmentation fault when deleting the	Pointer reassignment failed to check if	Added `if(current->p`	`deleteOrd er()` safely
	head or tail of the orders ledger.	`current->prev` or `current->next` were `nullptr`.	`rev)` and `if(current->n ext)` boundary checks before re-linking adjacent nodes.	removes extreme boundary nodes without crashing.
5	Memory leak warnings upon system exit during early testing.	Program was relying on default destruction; dynamic nodes were left on the heap.	Built explicit `~ClassName()` destructors with traversal loops that call `delete` on every node.	Zero memory leaks verified upon program termination.

INDIVIDUAL CONTRIBUTION MATRIX

Member Name	Assigned Role	Modules Developed	Testing & Documentation	Contribution %
Jan Martin Fermin	Lead System Architect Developer	Core Data Models & Masters Orders Ledger	Master Ledger CRUD Tests, System, Architecture Diagram	20%
Philip Gotinga	Queue Systems Developer	Priority Dispatch Queue Module	Queue Dispatch Logic Tests, Data Flow / Program Flowcharts	20%
Xander Macaraeg	Stack & Audit Developer	Checkout Cart & Coupon Stack	LIFO Undo Verification Tests, Bug & Debugging Log	20%
Mark Allen Tuscano	Algorithmic & Metrics Developer	Searching & Elementary Sorting Instrumentation	Algorithm Sorting Verification, Big-O Complexity Analysis	20%
Jesslyn Arsenal	Benchmarking Lead	Benchmarking Suite Engine	Performance Execution Log, Benchmark Results & Charts	20%

GITHUB AND DELIVERABLE CHECKLIST

PROJECT DELIVERABLES CHECKLIST

- Verify all required physical and digital submission assets:
- Complete C++ Source Code (.cpp, .h files)
- Active GitHub Repository
- Professional README.md file in repository
- System Technical Documentation
- Software Architecture Diagram
- Program Flowchart / Data Flow Diagram
- System Application Screenshots
- Benchmark Results Log & Charts
- Big-O Complexity Analysis Section
- Completed Test-Case Results Log
- Bug & Debugging Log
- Completed Individual Contribution Matrix
- Presentation Slides / Demonstration Materials
- Compiled Executable (where applicable)

GitHub Repository URL: https://github.com/notivee/CC105_Scenario9_Ecommerce
Latest Commit Hash: 909bf4f

Thank You